

An Analytical Study of Algorithmic Solutions for *Lights Out* and *Tetravex*

Ghulam Junaid

Abstract

This paper presents an analytical study of two algorithmic solutions developed for classic combinatorial games: *Lights Out* and *Tetravex*. The *Lights Out* solution uses a breadth-first search (BFS) over a bitmask-encoded state space to guarantee optimality, while the *Tetravex* solution is formulated as a constraint satisfaction problem (CSP) and solved using depth-first search (DFS) with backtracking, constraint propagation, and heuristic ordering. We analyze correctness, time and space complexity, design decisions, limitations, and scalable improvements of both approaches.

1 Introduction

Combinatorial puzzles provide a natural setting for studying state-space search, optimization, and constraint reasoning. Despite simple rules, many such puzzles induce exponential search spaces and non-trivial structural constraints. As a result, they serve as compact but expressive benchmarks for algorithmic ideas.

Lights Out is a deterministic finite-state puzzle with a well-defined transition graph, making it suitable for exhaustive shortest-path algorithms. *Tetravex*, in contrast, is a tile-placement puzzle characterized by strong local constraints and factorial growth of possibilities, requiring aggressive pruning and heuristics.

This paper analyzes two implementations reflecting these differences: a global BFS-based solver for *Lights Out* and a heuristic-driven DFS solver for *Tetravex*. The focus is not only on implementation, but also on modeling choices and structural improvements that significantly affect scalability.

Game References and Online Resources

For completeness, the interactive versions of the puzzles analyzed in this paper are available online:

- *Lights Out*:
<https://www.logicgamesonline.com/lightsout/>
- *Tetravex*:
<https://smart-games.org/en/tetravex/game/3>

These links provide a concrete reference to the game rules and mechanics used throughout the analysis.

Implementation Availability

The complete implementations for several of the ideas discussed in this paper are publicly available at:

<https://github.com/GhulamJunaid07/Lights-out-and-Tetravex-solvers>

2 Lights Out

2.1 Problem Modeling

The standard 5×5 *Lights Out* board is encoded as a 25-bit integer. Each bit represents the on/off state of a cell. Clicking a cell toggles the state of that cell and its four orthogonal neighbors (when they exist). This induces an implicit graph where:

- Vertices correspond to all 2^{25} possible board configurations.
- Edges correspond to a single cell click.

The objective is to reach the all-zero configuration from a given initial state.

2.2 Algorithm

The solver performs a breadth-first search starting from the zero configuration. For each state, all 25 possible clicks are explored by applying bitwise XOR operations to generate neighboring states. Distances and parent pointers are stored to reconstruct an optimal solution sequence.

BFS is particularly appropriate because the graph is unweighted and all moves have equal cost. The first time a configuration is visited, it is guaranteed to be reached via a minimum-length sequence of moves.

2.3 State Transition Implementation

The following code fragment illustrates how a single click transition is implemented using bit manipulation. The index i denotes the clicked cell.

Listing 1: Bitmask-based state transition in Lights Out

```
1 int u = v;
2 // v is current state, u will be the next state
3 u ^= (1 << i);
4 if (i < 20)
5     u ^= (1 << (i + 5));
6 if (i > 4)
7     u ^= (1 << (i - 5));
8 if (i % 5 != 4)
9     u ^= (1 << (i + 1));
10 if (i % 5 != 0)
11     u ^= (1 << (i - 1));
```

This constant-time transition function avoids coordinate arithmetic and makes the BFS efficient in practice.

2.4 Correctness

The algorithm is correct because BFS explores states in non-decreasing order of distance, all reachable configurations are eventually visited, and transitions are deterministic. If a configuration is unreachable, the algorithm correctly reports that no solution exists.

2.5 Complexity Analysis

Time Complexity. There are 2^{25} possible states, and each state generates 25 transitions: $O(25 \cdot 2^{25}) = O(2^{25})$.

Space Complexity. Distance and parent arrays are stored for every state: $O(2^{25})$.

2.6 Strengths and Limitations

This approach guarantees optimal solutions and allows fast query resolution after preprocessing. However, memory usage grows exponentially and limits the approach to fixed board sizes such as 5×5 .

2.7 Row-Reduction Approach for Larger Boards

A scalable extension is to reduce the effective state space by eliminating rows sequentially. Starting from the top row, moves can be chosen to turn off all illuminated cells in the current row, propagating their effects only to the row below. Repeating this process reduces any configuration to one where all active cells lie in a single designated row.

After this reduction, the entire board state is determined by the final row, yielding only 2^n possible configurations for an $n \times n$ board. This reduces the problem from 2^{n^2} states to 2^n states and is closely related to Gaussian elimination over $\mathbb{F}_2$.

3 Tetravex

3.1 Problem Modeling

Tetravex consists of an $n \times n$ grid and n^2 unique tiles. Each tile has four labeled edges (top, right, bottom, left). A valid solution places all tiles such that adjacent edges match exactly. This problem can be modeled as a constraint satisfaction problem with spatial and combinatorial constraints.

3.2 Algorithm

The solver uses depth-first search with backtracking. Tiles are placed starting from the center of the board and expanding outward. At each step, feasible tiles are filtered based on already placed neighbors, and the algorithm recursively explores valid placements.

3.3 Local Constraint Checking

The following function checks whether a tile can be placed at a given position without violating local constraints.

Listing 2: Local constraint validation in Tetravex

```

1 bool valid(int x, int y, int v) {
2     for (int d = 0; d < 4; d++) {
3         int nx = x + dx[d];
4         int ny = y + dy[d];
5         // (nx, ny) is an adjacent of (x, y)
6         if (!inside(nx, ny) || !mat[nx][ny])
7             continue;
8         // (nx, ny) is inside the grid &
           assigned
9         if (vec[mat[nx][ny]][(d + 2) % 4] != vec
10            [v][d])
11             return false;
12         // must have the same number but in
           opposite direction
13     }
14     return true;
15 }
```

By enforcing constraints immediately, invalid partial configurations are pruned early, dramatically reducing the effective search space. Assigning the directions in clockwise order (Left, Up, Right, Down) was a trick in implementation to have the +2 index as the opposite direction, the code successfully uses this trick to check the condition.

3.4 Complexity Analysis

Time Complexity. In the worst case, the time complexity is factorial: $O((n^2)!)$.

In practice, constraint propagation and heuristic ordering significantly reduce runtime.

Space Complexity. The algorithm uses $\mathcal{O}(n^2)$ memory for the board representation, recursion stack, and auxiliary structures.

3.5 Row-Based Decomposition

An improvement is to decompose the board row-wise. All valid row configurations are first generated using a DFS that enforces horizontal constraints. A second DFS then selects a compatible ordering of these rows based on vertical edge matching. This reduces recursion depth from n^2 to n and strengthens constraints early in the search.

4 Author’s Insight and Lessons Learned

Developing these solvers emphasized that modeling choices often dominate raw computational power.

Structural reformulations—such as reducing two-dimensional constraints to one-dimensional layers or exploiting algebraic structure—can yield exponential improvements. These lessons generalize well beyond the puzzles studied here.

5 Conclusion

The two solutions illustrate contrasting but appropriate algorithmic paradigms. Lights Out benefits from exhaustive state-space traversal and algebraic structure, while Tetravex relies on heuristic-guided search and constraint reasoning. Together, they demonstrate how principled modeling transforms infeasible brute-force approaches into practical solvers.

Citations

- *Lights Out Puzzle*, Tiger Electronics, 1995.
- *Tetravex*, Roger von Oech, 1988.
- *Introduction to Algorithms*, T. H. Cormen et al., MIT Press, 2009.
- *Linear Algebraic Methods for Lights Out*, A. Berkove et al., American Mathematical Monthly, 2000.
- *Constraint Processing*, R. Dechter, Morgan Kaufmann, 2003.